

Compte rendu Développement du jeu Shifumi

Sommaire

Introduction

- [1. Définition des règles du jeu](#)
- [2. Déclaration et initialisation des variables](#)
- [3. Choix du nombre de points pour la partie](#)
- [4. Choix du joueur](#)
- [5. Choix aléatoire de l'ordinateur](#)
- [6. Détermination du gagnant de la manche](#)
- [7. Affichage du score](#)
- [8. Fin de la partie](#)
- [9. Proposition de rejouer](#)

Conclusion

Introduction

Le projet présenté consiste en la réalisation d'un jeu de Shifumi (Pierre-Feuille-Ciseau) en langage Java. L'objectif est de permettre à un joueur d'affronter l'ordinateur selon les règles classiques : Pierre bat Ciseau, Feuille bat Pierre, Ciseau bat Feuille et en cas de choix identiques, il y a égalité. Ce compte rendu décrit les différentes étapes de conception et de codage de ce programme.

1. Définition des règles du jeu

Il est important de rappeler les règles du jeu au joueur. Ainsi, dans le code, ces règles sont affichées dès le lancement du programme à l'aide d'instructions `System.out.println()`.

- Feuille bat Pierre
- Pierre bat Ciseau
- Ciseau bat Feuille
- Égalité si les choix sont similaires

```
import java.util.Scanner;
/*
 * Classe principale pour le jeu shifumi(pierre-papier-ciseau)
 * ce programme initie un jeu contre l'ordinateur avec des règles class:
 *
 * @author Pedanou Houéfa
 * @version 1.0
 */
public class tp4shifumi {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        System.out.println("Voici les règles du jeu");
        System.out.println("Feuille bat Pierre");
        System.out.println("Pierre bat Ciseau");
        System.out.println("Ciseau bat Feuille");
        System.out.println("Égalité si 2 éléments similaires");
    }
}
```

2. Déclaration et initialisation des variables

Plusieurs variables sont nécessaires au bon déroulement du jeu :

- nbdeponts : nombre de points nécessaires pour gagner la partie (3, 5 ou 10).
- choix : choix du joueur (P, F ou C).
- choixordi : choix aléatoire de l'ordinateur.
- aleatoire : entier généré pour simuler le choix de l'ordinateur.
- scorechoix et scoreordi : compteurs de score du joueur et de l'ordinateur.
- reponse : réponse du joueur pour décider de rejouer ou non.
- rejouer : variable booléenne qui gère la boucle de répétition des parties.

L'objet Scanner sc est utilisé pour gérer les entrées clavier du joueur.

```
//Variables
int nbdeponts;|
char choix;
int aleatoire;
char choixordi;
int scorechoix=0;
int scoreordi=0;
char reponse;
boolean rejouer = true;
Scanner sc = new Scanner(System.in);
```

3. Choix du nombre de points pour la partie

Le programme propose au joueur de définir le nombre de points nécessaires pour remporter la partie (3, 5 ou 10). Une boucle **do ... while** vérifie que l'entrée est valide.

```
do {
    System.out.println("En combien de points se déroule la partie ?(3,5
    nbdeponts = sc.nextInt();
    System.out.println("Vous avez saisi :"+nbdeponts);
}while (nbdeponts!=3 && nbdeponts!=5 && nbdeponts!=10);
```

4. Choix du joueur

Lors de chaque manche, le joueur doit entrer son choix parmi :

- P pour Pierre
- F pour Feuille
- C pour Ciseau

Une boucle **do ... while** garantit que le choix saisi est valide.

```
do {
    System.out.println("Choisir Pierre(P), Feuille(F) ou Ciseau(C)");
    choix=sc.next().charAt(0);
    System.out.println("Vous avez choisi:"+choix);
}while (choix!='P' && choix!='F' && choix!='C');
```

5. Choix aléatoire de l'ordinateur

L'ordinateur choisit son coup de manière aléatoire grâce à la fonction **Math.random()**. Le nombre généré (entre 1 et 3) est associé respectivement à Pierre, Feuille ou Ciseau.

Un délai (**Thread.sleep(3000)**) est inséré pour ajouter du suspense avant de révéler le choix de l'ordinateur.

```
aleatoire= (int)(Math.random()*(3))+1;
if(aleatoire == 1) {
    choixordi='P';
}
else if (aleatoire == 2){
    choixordi='F';
}
else {
    choixordi='C';
}

System.out.println("l'ordinateur a choisi...");
//étape 4: révélation du suspense

//try catch permet de continuer le programme en cas de problème de f
try {
    Thread.sleep(3000);
}catch(InterruptedException e) {
    e.printStackTrace();
}
System.out.println("l'ordi a choisi:"+choixordi);
System.out.println("Fin du suspense!");
```

6. Détermination du gagnant de la manche

Les règles de comparaison sont appliquées :

- Si les deux choix sont identiques → égalité.
- Sinon, on détermine si le joueur gagne selon les règles

classiques.

- Le score du gagnant est incrémenté d'un point.

```
if (choix == choixordi){
    System.out.println("Egalité, aucun point");
}
else if
    ((choix == 'P' && choixordi == 'C') ||
     (choix == 'F' && choixordi == 'P') ||
     (choix == 'C' && choixordi == 'F')){
    System.out.println("Vous avez gagné!!!");
    scorechoix++;
}
else {
    System.out.println("Vous avez perdu.");
    scoreordi++;
}

System.out.println("Score: Joueur: "+scorechoix+" -Ordinateur: "+scoreordi+"")
```

7. Affichage du score

Après chaque manche, le programme affiche le score actuel sous la forme :

Score: Joueur: X - Ordinateur: Y

```
System.out.println("Score: Joueur: "+scorechoix+" -Ordinateur: "+scoreordi+"")
```

8. Fin de la partie

Une fois qu'un des deux participants atteint le nombre de points requis :

- Si le joueur gagne → message de félicitations.
- Si l'ordinateur gagne → message de défaite.

```

// Etape 7 : rejouer la partie
if (scorechoix == nbdeponts) {
    System.out.println("Félicitations! Vous avez gagné la partie!");
} else {
    System.out.println("L'ordinateur a gagné");
}

```

9. Proposition de rejouer

Le jeu intègre une fonctionnalité de rejouabilité permettant d'améliorer l'expérience utilisateur. À la fin d'une partie, le programme propose au joueur de recommencer une nouvelle partie en répondant par 'O' (Oui) ou 'N' (Non). Cette interaction est gérée par la variable '**reponse**' qui enregistre le choix du joueur.

Si le joueur répond 'O', la variable booléenne '**rejouer**' reste vraie et la boucle principale reprend, tout en réinitialisant les scores à zéro pour garantir des conditions équitables. Le joueur peut ainsi redéfinir le nombre de points nécessaires pour la partie suivante.

Si le joueur répond 'N', la variable '**rejouer**' est passée à false, ce qui interrompt la boucle principale. Le programme se termine alors proprement en affichant un message de remerciement et d'au revoir. Cette fonctionnalité permet au joueur de décider lui-même de la durée de son expérience de jeu.

```
do {
```

```

    System.out.println("Voulez-vous rejouer?(O/N)");
    reponse = Character.toUpperCase(sc.next().charAt(0));
    if (reponse != 'O') {
        rejouer = false;
    }
    System.out.println("Merci d'avoir utilisé le programme. Au revoir");
    scorechoix=0;
    scoreordi=0;
}
while (rejouer==true);
System.out.println(" Le Jeu va recommencer");
sc.close();

```

Conclusion

Ce projet a permis de mettre en pratique :

- L'utilisation des boucles (do ... while, while).
- La gestion des conditions pour déterminer les résultats.
 - L'utilisation d'un générateur de nombres aléatoires pour simuler l'adversaire.
- La gestion des entrées utilisateur avec Scanner.
- Une approche interactive, avec un système de rejouabilité.

Ainsi, ce programme illustre de manière simple et efficace la conception d'un jeu classique en Java.